SCORE: A Framework for Synthesizing Complete, Owned, Responsive Environments

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 11 October 2026

Abstract

Games, films and robot training need 3D worlds that are complete, editable and owned by the people who make them. World models and one-shot 3D generators can produce a convincing place from a sentence or a picture, but they give frames that no engine can open, or one fused scene without separate objects or clear rights. Studios have always built worlds in stages, from concept art and layout to modelling, surfacing, sound and review. We present SCORE, a world generation framework that keeps these stages and automates large parts of them with open vision foundation models, procedural tools and coding agents, while the creator chooses and reviews. Because every stage's output is kept, the creator can rerun or fix any single stage, and because every model allows commercial use, the creator stays the author of the result, holding its full commercial rights. We are building SCORE's first world, the game world of 2099, whose places do not yet pass their own completion gate, and plan three further worlds of different kinds. Our aim is to show that open models, working in checkable stages, can build worlds that a creator owns and keeps editing. Code and paper are available at https://github.com/Babon-Innovations-b-v/score (MIT license).

Gap: teaser figure. This figure will show four worlds made with SCORE: the game world of 2099, a stylised underwater world, a simulation world for robots, and a third-person fantasy world. Each will appear as its chosen concept beside the finished world in its engine. It waits until the four worlds are finished and accepted by their creators.

1 Introduction

Interactive 3D worlds are the raw material of games, of film and of embodied AI, where agents are trained and tested in simulators [1]. Building such a world is still expensive. Every object has to be modelled, given surfaces, placed, given collision and physical properties, lit and given sound, all in one consistent style, and the world has to stay editable while the project changes. Generative models promise to take over much of this work, but the people who would use them are wary of it: in GDC's 2026 survey, 36 percent of game workers used generative AI at work, and 52 percent thought it was hurting the industry [2]. Taking over the work is therefore not enough: a tool for building worlds has to leave the result in the creator's hands.

Two lines of research generate the world itself. The first, learned world models, generates it as frames. Genie learns an interactive environment from video [3], and Genie 3 renders a world that a person can move through in real time; its announcement lists a constrained range of actions and a few minutes of continuous interaction among its limits [4]. Nothing that these models produce can be opened in a game engine or an editor, and their world exists only as far as the player looks. The second line, 3D world generators, produces geometry instead. WorldGen turns a text prompt

into a traversable scene for standard game engines [5], and HY-World 2.0 turns text or a picture into a single splat or mesh world [6]. Most such outputs, however, are "static monolithic assets with limited editability and physical interaction" [7], and WorldAct and WorldSculpt address this by recovering separate objects from them after the fact [7], [8]. In both lines the world is generated in one shot. When part of the result is wrong, the creator can regenerate the whole world or repair it by hand, but cannot see which step caused the fault, and cannot change that step alone.

A third line of work keeps the world explicit. Infinigen and Infinigen Indoors build every asset from procedural rules, with materials as generators of their own [9], [10]. This gives real geometry and full control, but only over the content that the rules describe. Holodeck has a language model write layout constraints, which a solver then satisfies with assets retrieved from a library [1]. SceneCraft, recursive code world models, WorldClaw and AutoUE let coding agents write scenes, or whole games, as programs [11], [12], [13], [14]. LEGO-Anything shows what such agents need. When coding agents rebuild a scene from a single picture, they start weakly, undo their own progress while editing, and judge their own geometry unreliably; giving them grounded tools, rather than training them further, closes much of that gap [15]. All of these systems, however, either rebuild a scene that already exists or draw their objects from a fixed library of assets.

Meanwhile, each of the parts that a world needs can now be read out of pictures by an open model. Vision foundation models learned the world from images in the way that language models learned it from text [16]. They can now turn a picture into a 3D shape aligned with its pixels [17], into a mask for each named object [18], into depth in metres [19], [20], into the parts of an object [21] or into a human body [22]; related models turn a sentence into motion [23] or into sound [24]. Most of these models pass on pictures, masks, depth maps or meshes rather than their internal features, so a coding agent can connect them to one another as it would connect tools. Chorus goes the other way and distils several 2D foundation models into one encoder of 3D Gaussian scenes, whose features serve segmentation and question answering [25]. It reads a scene that already exists, while SCORE builds one, and SCORE joins its models only through what they output, so that any of them can be replaced. What none of them provides is one consistent style across objects that different models have made. The field names this as an open problem [26], [27], and Hunyuan3D Studio addresses it only by restyling the input picture [28].

Studios have always built worlds in stages: concept art, layout, asset lists, modelling, surfacing, sound and review. Each stage has a clear output that a person can check before the next one starts, which makes this way of working the natural basis for a world generation framework. SCORE keeps these stages and automates large parts of them with the models above, with procedural tools and with coding agents, while the creator chooses and reviews. Its name states what it aims for. The worlds it makes are **complete**: every object, surface, light and sound exists wherever the player looks, and everything that the world's core mechanics need, such as collision, the doors that open and the sounds that play with them, is made when the world is built rather than added later. They are **owned**: the creator decides what the world is and stays the author of the result, holding its full commercial rights, because every model in the framework allows commercial use of its output. They are **responsive**: the world answers to the physics engine that loads it, because every object has real collision, mass and material, so it behaves physically in any engine or simulator, under physics that each world may set for itself, such as the low gravity of the Moon. And they can be any kind of **environment**, real or made up. SCORE does not compete with procedural tools, part splitters or world generators; it takes them in as building blocks wherever their licences allow. It also does not generate a world in one shot: the creator chooses the style, the concepts and the storyboards up front and reviews each stage, and coding agents and models do the work of the stages between

those choices.

A world made this way is meant to come whole. In world 1, people are built with SAM 3D Body and move by clips that Kimodo writes from sentences, and doors and a rocket move at the speeds of the game the world was made for; the creator has reviewed none of these layers yet, and he rejected the people's clothes (Limitations). A place is one OpenUSD stage, which opens in the tools that artists and engineers already use. A creator drives the framework with a coding agent's help rather than by learning each of its tools, and every heavy stage runs on rented cards that are deleted after each batch, or from pinned container images on any Kubernetes cluster, a path not yet used for world 1. To our knowledge, no other openly available world generation work joins vision foundation models into engine-ready scenes that are built from reference pictures, steered by a person's choices at each step, and made only with models that allow commercial use of their output.

This paper makes three contributions:

- **The SCORE framework:** staged, retraceable generation from a creator's choices to one canonical scene, which consists of a generated base layer and the creator's own edit layers (Section 2).
- **The mechanisms that keep its worlds coherent and make faults cheap to fix:** deterministic checks before every stage that costs money, a parts check that decides how each object is built so that every real-world object becomes one model, a shared library of rule-based surfaces that keeps shape and surface apart, and review tools for the creator (Section 2).
- **Evidence from four worlds of different kinds:** the time, the cost and the faults caught before spending at every stage, and an evaluation on a shared benchmark (Sections 3 and 4).

2 Methods

Figure 1 shows the stages on one place. The creator chooses a concept (a), and a coding agent turns it into a dimensioned plan (b) and a scene inventory (c). An open picture model draws a close-up of every object in the inventory, and a closed one redraws those that fail a shape check (d). Each object is then built in code or generated in 3D, its parts are painted from the shared library of rule-based surfaces (e), and the place is assembled in an engine (f). The output of every stage is kept, so the creator can trace a weak result to the stage that caused it, rerun or fix that stage, and regenerate the stages after it while keeping their own edits. The sections below describe each stage with an example. Every stage that loads a model runs on rented cards, on single machines, or as a job in a pinned container image on any Kubernetes cluster (Appendix G).

2.1 Inventory as a partition of the concept

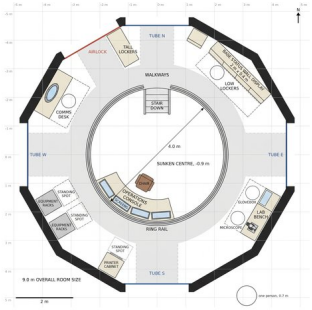
The inventory decides what exists in a place. A coding agent cuts the concept into tiles and lists every visible element, and each element either becomes a row (with its kind, anchor, size and count) or is dropped with a written reason (Figure 2). Nothing is made or placed that is not a row. After the build, the place is rendered from the concept's own camera and compared with the concept; a place that is clearly sparser than its concept fails this concept-density check.

2.2 Close-ups: the open model first

Every object is built from a close-up: a picture of the object alone on a plain background, drawn from its crop of the concept. Qwen-Image-Edit-2511 [29], an open model that runs on our own



(a) concept, picked by the creator



(b) dimensioned plan

scene inventory (excerpt)

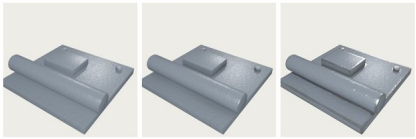
id	anchor	size	count
console	floor	2.6x0.6x1.2 m	x1
labbench	floor	2.0x0.7x1.5 m	x1
chair	floor	0.6x0.6x0.9 m	x1
talllocker	floor	1.2x0.55x2.0 m	x1
rack	floor	0.6x0.6x1.9 m	x2
comms	floor	1.5x0.7x1.2 m	x1
toolboard	wall	1.2x0.08x0.8 m	x1

... 74 rows in all

(c) inventory



(d) close-ups

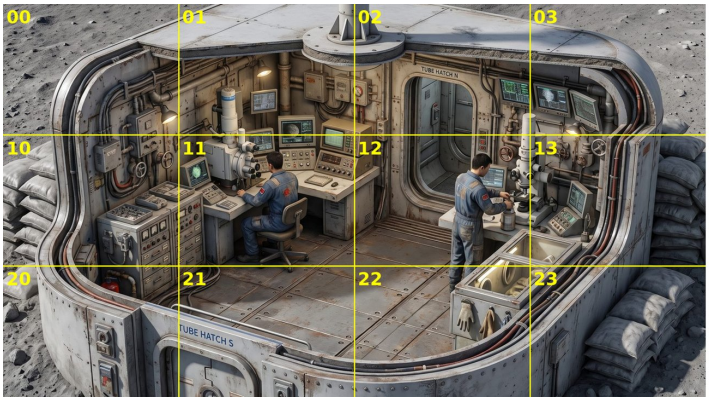


(e) library surface, three wear levels



(f) assembled room in the engine

Figure 1: The stages on one place, the central hub of the game 2099. (a) The concept, drawn by a picture model from the creator's reference pictures and chosen by the creator. (b) The dimensioned plan in metres, with walkways and standing spots. (c) An excerpt of the scene inventory, which has 74 rows. (d) Close-up pictures of two objects. (e) One surface from the shared library of rule-based surfaces at three wear settings. (f) The assembled room in the game engine.



each element of the concept -> an inventory row

tile 11 01 13: research microscope
  -> microscope
tile 12 22: glovebox cabinet
  -> glovebox_cabinet
tile 10 12 13: valves on the gas lines
  -> pipe_valve
tile 10: fire extinguisher
  -> fire_extinguisher

tile 02 03: three wall monitors
  -> wall_monitor
tile 21: coil of cables on the floor
  -> dropped: loose cable on a walkway floor
tile 12: petri dish, small tools
  -> dropped: under 10 cm, unreadable from...
tile 11 12: two crew at the desks
  -> dropped: people are the crew's, not the room's
47 elements; 7 dropped with a reason

Figure 2: The inventory of the lab on the Moon base, built from its concept. Left: the concept, cut into twelve tiles. Right: examples of how each visible element maps to an inventory row, or is dropped with a written reason. Of the 47 elements in this concept, 40 were mapped to inventory rows and 7 were dropped with a written reason.

rented cards, draws every close-up first. A shape check then decides, without a person, whether the picture may go on to the 3D step. Its first half measures the picture: a plain border all round, one object filling a sensible share of the frame, and an outline in the proportions of the inventory row's box. Its second half is an open vision-language model, Qwen3.8-27B [30], which sees the crop beside the close-up and answers set questions (one object, a clean background, the whole object, the same object, no parts missing or added, the right proportions); each question is asked three times with different seeds and the majority decides. Only a close-up that fails is redrawn by Nano Banana Pro [31], first from the crop alone and then with the whole concept beside it, and its pictures pass the same check. The check leans towards failing, because a wrong picture that passes costs a bad 3D model, while a good one that fails costs only a picture from the closed model (Appendix E).

2.3 Code or generation: the parts check

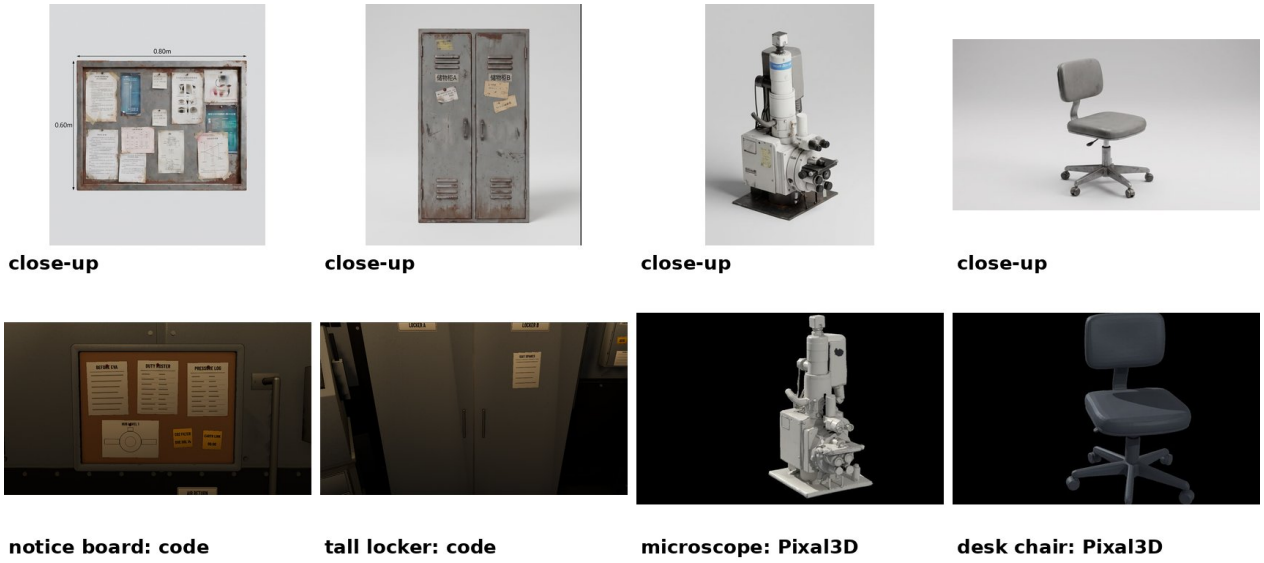


Figure 3: The parts check decides how each object is built. Top: the close-up picture of four objects. Bottom: the object as built. The notice board and the tall locker are built in code, because their code builds show every part of their close-ups. The microscope and the desk chair are generated with Pixal3D, because no code build shows all of their parts.

Picture-to-3D models build chunky solids well, but flat panels, thin beams and openings badly, while code builders are exact and straight but show only the parts that the coding agent wrote. For each kind of object, the parts check renders a code build from the front and compares it with the close-up picture: if every part of the close-up is visible in the build, the kind is built in code, and otherwise it is generated from the close-up with Pixal3D [17] and made solid (Figure 3). Composite objects are split in the inventory, so that every real-world object becomes one model: a console, for example, becomes a desk carrying separate monitors and keyboards.

2.4 Shape and surface apart

Models and code builders give shape only. Every part of every object takes one surface from a shared library of rule-based surfaces, written as ProcFunc functions [32] that are built on Infinigen's shaders [9] and take their colours from the place's palette (Figure 4a). A room has one wear setting, and wear is applied where it has a cause, such as edges and the places where feet pass. On a generated



(a) library surfaces at wear 0, 0.4, 0.8 (b) tool board: round four (left), round six (right)

Figure 4: (a) Three surfaces from the shared library of rule-based surfaces (painted panel, bare steel and deck plate) at wear settings 0, 0.4 and 0.8. (b) The hub's tool board when each generated piece kept the colours of its own picture (left), and after every surface came from the library and each tool became its own model (right). Both shots use the same camera and the same build of the game.

model, PartCrafter [21] splits the shape into parts, and an open vision-language model, shown each part outlined on the close-up, names the library surface it takes; the picture's colours are not kept. Figure 4b shows why: when each piece kept the colours of its own picture, every piece brought its own rust and stains, and the room looked inconsistent. This made the hub consistent, and the creator accepted it (Table C1). On the rest of world 1 the painting is not yet consistent: 42 of its 129 stored takes still carry an early colour pick per PartCrafter part, later takes are split along SAM 2.1 regions of the close-up before the judge names each surface, and that method still fails four of six named models, because one part can hold two finishes and the judge reads shadow as dark paint (Limitations). Sound is attached in the same way, to surfaces, rooms and objects, and is generated by MOSS-SoundEffect [24] (Appendix F).

2.5 Terrain

Large outdoor places follow the same pattern (Figure 5). A dimensioned plan places the flat areas and paths the place needs (a), a diorama picture drawn over the plan sets the landforms (b), and the heights follow the picture (c). A skin is painted over the plan, with fine relief from monocular depth [20] (d), rocks are single generated models, and everything is built on the world's own curved ground (e).

2.6 Checks before spending

Deterministic checks run before every stage that costs money. They test real geometry rather than relying on an agent's judgement, because a fault costs nothing to fix in the plan but an evening of the creator's time once the place is built. The checks look for light leaking out of the room, pieces in front of openings or off their surface, doors that do not separate their two sides, models that are open, too thin or leaning, and surfaces off the palette (Appendix D). A room whose shell is laid as a kit of made pieces, such as the hub, is scored as a partition of its surfaces, after Point2Part's requirement that parts neither overlap nor leave gaps [33]: every visible surface should belong to exactly one piece, and the score gives the share that pieces cover, the share that two pieces cover and the largest patch left bare. Figure 6 shows one fault the placement check caught. A failed check sends the work back one stage.

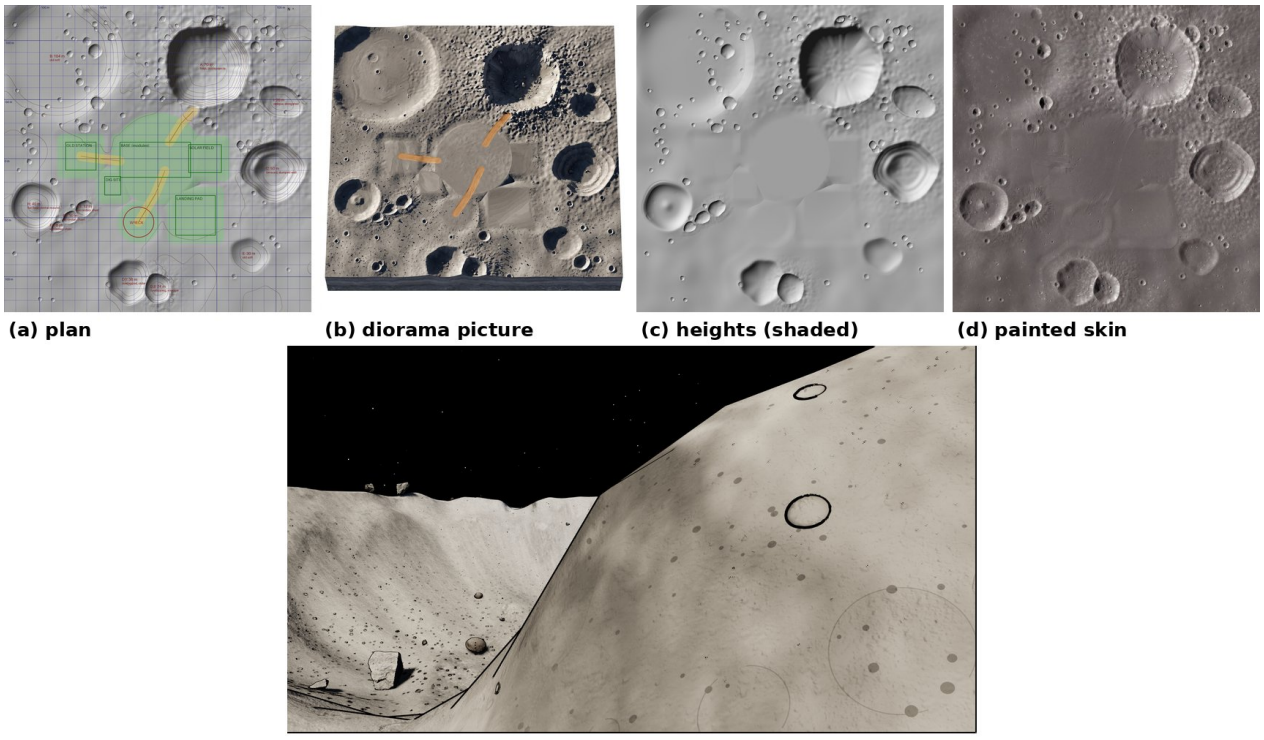


Figure 5: The stages for large outdoor places, on the ground around the Moon base. (a) The dimensioned plan with the base's flat areas and paths. (b) A diorama picture drawn over the plan. (c) The heights that follow the diorama, shown shaded. (d) The skin painted over the plan. (e) The result in the engine, seen from a crater rim.

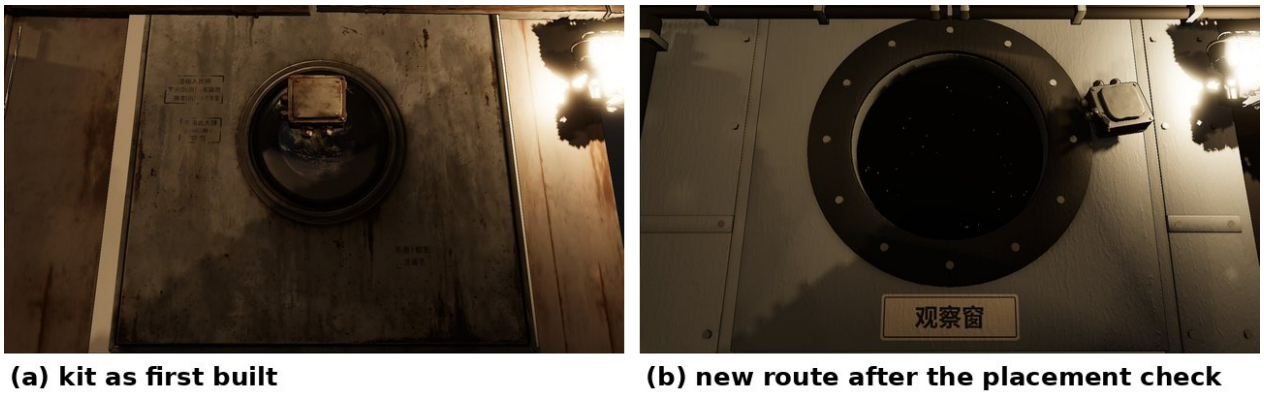
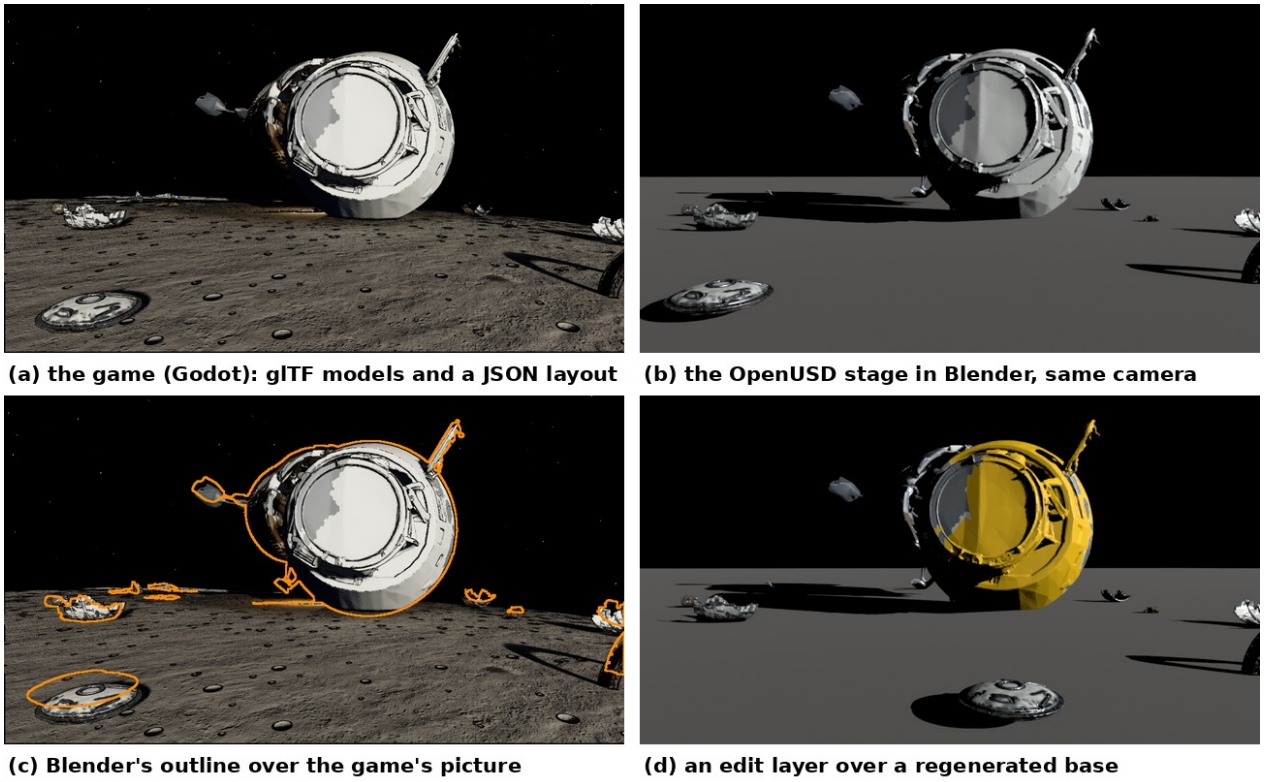


Figure 6: A fault caught by the placement check. (a) In the first build of the hub, a conduit box covered the porthole. (b) In the current framework, the placement check found the box in front of the opening and moved it 0.5 m along the wall before anything was spent.

2.7 The canonical scene

Every place is exported as one OpenUSD stage [34] in which each object carries its kind, its transform in metres, its collision, the library surface of each part and that surface's sounds, with the generated base layer under an edit layer that belongs to the creator, so that an edit survives when the base is generated again (Figure 7). A test place checks this across a base that changed; on a real place it is shown only for a base generated again, not yet for one whose models changed, such as a repainted model.



(a) the game (Godot): glTF models and a JSON layout

(b) the OpenUSD stage in Blender, same camera

(c) Blender's outline over the game's picture

(d) an edit layer over a regenerated base

Figure 7: The wreck, one outdoor place of the game 2099, as an OpenUSD stage. (a) The place in the game, from one of its fixed cameras. (b) The stage loaded in Blender and rendered from the same camera. (c) Blender's object outlines drawn over the game's picture. (d) The same camera after an edit in the edit layer, made here by a script, that moved the round cover and gave the capsule's painted hull another library surface, and after the base layer was generated again.

A place's stage holds more than its inventory's objects. A scene record adds what the place needs around them, each piece built by a code builder and painted from the library: the room's shell and stairs, the ground out to the horizon, water, the far backdrop, the neighbouring places seen through open doorways (each referenced as its own stage), every light with the sky and the exposure, and the cameras that the place is judged from.

Before a place is accepted, its loose objects are dropped in a physics simulation on the stage's own ground with their own triangles, and each rest pose is written back into the layout. Settling only corrects: an object that would turn or drift further than a small limit keeps its laid pose and fails the resting check, which also fails anything floating, tipping, sunk into the ground or lying inside another piece (Appendix D). A large move is a fault of the layout, not of physics, and the layout follows the creator's concept. On the wreck's stage of 8 October, 11 of 22 pieces rested as first laid;

the coordinating agent then laid the wreck out again after its concept and cut shallow dents in the ground where pieces had struck it, and all 22 rested. On the old station's stage of that day, the check found a pile of sandbags lying inside the lander's legs, and once the pile was moved, all 29 pieces rested. Both places have changed since the creator reviewed them, and on 10 October both failed the completion gate.

A place is complete only when its evidence says so. Coding agents judging their own scenes from renders agree with a measured score on geometry only 45.8 percent of the time, below chance [15], and ours called places finished that the creator then found unfinished. SCORE therefore computes completeness instead of asking for it. A completion gate passes a place only when every inventory row has its object on the stage or a written reason for dropping it, nothing visible stands in for a made piece, no real resting fault is left, every model the creator picked is still the one shown, every check was run on the stage as it is now, and the review page was rendered from that stage. Every check answers pass, fail or unknown, and unknown blocks like fail, as in LEGO-Anything's validator; a hook in the agent's harness refuses to end a session that claims a place is done while its gate does not pass. The gate is only as good as its checks. The first placeholder check judged only plain primitives, so the hangar's ship, which a code builder assembled from a hull and legs, passed it until the creator saw it as a placeholder; the check now judges every mesh a code builder draws, and only the shell builders pass unnamed. A reach check, in which a walker on the place's floor must reach every row, runs beside the gate but is not counted, because it fails backdrop rows that nobody walks to. On 10 October, 3 of the 18 places the gate judged passed, 14 failed and one was unknown. The resting check became a triage. Each object is judged by what holds it up, read from its inventory row (the floor, a wall, the ceiling, the object it stands on, a hook it hangs from, or the kit's own frame), with SceneEval's guards against false alarms [35]: an overlap counts only if it survives moving the object 5 mm away, and a standing object needs four contact points with its weight inside them. Cables, cloths and rubble may sink into what they lie on by an amount kept as data, and a loose object that moves more than 0.2 m or turns more than 8 degrees when dropped does not rest as laid, after SAGE's stability rule [36]. The real faults are grouped by cause, each group with one close-up aimed at its worst case. Over world 1's 17 places, the plain check flagged 4,078 of 12,032 objects; the triage kept 104 objects with real faults in 85 groups and marked the other 3,974 as false alarms, most of them kit pieces that stand where the kit lays them. The drop test ran on every place with loose pieces on its floor except the square, where it did not finish in three hours. These counts measure detection, not repair, and they were taken with the drop test of 8 and 9 October, whose faults (what lay on a dropped piece was treated as solid, and the test for a piece starting inside another was too strict) were corrected on 10 October; they have not been measured again. The creator's pick of a model is written onto each of its objects as the file's hash, and an export that would show anything else for a picked row is refused unless the creator's decision to replace it is written down. Finally, as a candidate check that no gate reads yet, each object is rendered alone from eight directions and compared with its close-up by its outline and by DINOv2 similarity [16], and the least similar are listed first: in the lab, the four lowest were a mouse drawn black, a ceiling lamp without its lens, a scanner without its screen and a wall monitor with a blank screen. The collision queries behind these checks follow the interface of ProcFunc's scene tools [32], whose code is not yet released, built on python-fcl and trimesh, and one annotation step gives every check per-object masks, depth, normals and occlusion boundaries from any camera, the same labels a robotics world needs.

A place's people are part of its stage. Its cast records who is there, how many, where, doing what and why each is there, and nobody is placed who is not in it. Each person becomes a skinned character (UsdSkel) that plays a clip from its own starting point, in a layer of its own between the

creator's edit layer and the base. A crowd is one point instancer of a cheap body, with a prototype for each clip, phase and palette of clothes. On the leader's walk, UsdSkel skins the converted body to within about a micrometre of the points the glTF file's own skinning gives. The prologue's square holds the leader on the podium, a front row of 24 people mixed from six kit builds, and a crowd of 10,000.

Characters are made by the framework's character maker, which has one entry point. A short spec gives a name and a picture or a few words, and the maker returns a rigged, clothed, textured and animated character as a glTF file and as UsdSkel, with a far body for crowds and a set of clips. A person's whole chain runs on one rented card. The A-pose picture is the creator's own, or FLUX.2 klein 4B [37] draws it from the words. SAM3DBody-cpp [38], a C++ and ONNX runner of SAM 3D Body [22], reads the body off the picture, and the body is fitted to SOMA-X through MHR. Kimodo [23] writes the clips from sentences. The clothes are GarmentCode's sewing patterns [39], draped on the body by the cloth solver of Newton [40], an open physics engine on NVIDIA Warp [41]. Hi3DGen [42] makes the hair from a clay picture of the bald head, and the hair is laid on the scalp as a shell. FLUX.2 klein base 4B [43] with a reference-depth LoRA [44] draws the face on the head's own depth view. The chain then builds the character, converts it to UsdSkel and renders it for review. Rebuilt from her own picture, Nev, a crew member of the Moon base, took 11.2 minutes of an H100, and a new person made from one sentence took 21 minutes (Appendix G). Animals use the same entry point with a route that does not assume a human body, still a trial that runs only when asked for: Pixal3D makes the mesh from a close-up, UniRig [45] rigs a quadruped, a fish gets a spine rig made in code, and the clips are made in code. GarmentCode's own solver, a fork of NVIDIA Warp, cannot be used, because its licence allows non-commercial research only. Newton's drapes bend between neighbouring cloth triangles about as Warp's do: on three bodies, the work suit's mean bend is 3.3 to 3.9 degrees, against 2.9 to 3.3 for Warp and 13.8 to 19.8 for Blender's cloth (Table G6). That measure missed what the creator saw: he rejected the Newton drapes as bloated at the chest and sleeves, and a later silhouette check, which compares each drape's width and stand-off with Warp's at dense cuts, fails them. Every crew and cast body was draped again with Newton, and the cast writer refuses a body whose drapes come from a simulator that does not allow commercial use; the drapes are now a trial, so the maker drapes a person only when its spec names the simulator. Kimodo stands people 1.6 to 2.2 hip widths apart, so the build brings the feet in to hip width; the hands run up under the sleeves and the trouser legs into the boots, and a check measures the wrists, ankles and stance on every clip of every body. One part is weak: the animals' clips are made in code, so the dog's paws twist and its feet slide (Limitations).

A place's sound is also part of its stage, in its own layer. Every sound is a UsdMedia spatial audio prim with its file, its gain and how it plays. The place's room tones loop without a position. An object's own hum or fan loops where the object stands. Each library surface points at its footstep, impact and scrape sounds, which an engine plays when something happens on that surface. The levels, buses and reaches are those the game 2099 played, now kept as the framework's sound catalogue. On the hub, the layer holds 2 room tones, 23 object sounds and 9 surface sounds. The review's walk plays what its camera hears along its path: each sound at its gain over its distance from the eye, panned by where it stands across the view, and the demo's video tool mixes the same track into each shot. Nobody has yet listened to this layer in a review.

What moves in a place moves in its stage as well, in a layer of its own: OpenUSD time samples on the prims that the base already holds, at the game's own speeds and lengths. The airlock's doors sink 2.8 m into the floor at 3 m a second, and its warning light burns while the pump empties the chamber and while the tanks fill it again, five seconds each. The garage's big door slides apart at

2.5 m a second under a warning beacon, and the hangar's roof leaves slide 11.2 m at 3 m a second. How long a door stands open is the player's choice in the game, so the motion record sets those pauses itself and says so. In the launch view, the rocket lifts off 3 seconds after its engines light and climbs at the game's rocket thrust of 24 m/s² against Earth's gravity. The game never draws this, because the player rides the launch inside the capsule, so the climb is an addition of the record, timed by the game's launch sequence. Each motion also lists the sounds that the game plays with it and when, and the sound layer plays them from the moving prim. The places' vehicles and robots do not move yet: no place of world 1 shows the game's rover, drone or robots, and the street's car stands still in the game as well.

A place's particle effects are a stage layer as well, and each emitter is written twice. One prim carries the effect's parameters in the game's numbers: its emitter shape, rate, lifetime, speed, spread, gravity, size and opacity over its life, the wind it follows and its seed, so that an engine can make the effect again with its own particle system. Under it, the particles are baked as points over a loop that repeats exactly, so any USD reader shows them without knowing those parameters. The bake repeats the particle rules of Godot, the game's engine, with the framework's own seeded draws, so the same record always gives the same particles. The creator's rule for effects is to start from real footage and to use many fine particles, because big or sparse ones read as random dots, and the records keep the game's counts. On the calm afternoon that the camp's scene record holds, 30,000 grains of 3 cm blow along the ground on 25 cells round the camp, and two dust devils of 6,300 particles each wander downwind at 1.4 m/s; a storm variant of the same layer holds 180,630 particles. The flat holds the wisp and the breath of the cigarette smoked on its balcony. Six of the game's ten kinds of emitter are placed. The dust thrown by a boot, a rover's wheel and a digger's scoop is not, because no place of world 1 holds the walker or the machine that throws it, and the landing dust left the wreck with the landing pad it was thrown across.

The framework stands apart from the game it was first built for. Every file a record names is in the repository or in one release of it: the first world's made models (199 MB packed) and its 121 sounds (36 MB), each listed with its checksum, its source and its licence. A check fails when any code or data names a file of the game's own tree.

Some surfaces need more than a flat library colour, so the framework writes its own shaders as UsdPreviewSurface materials over pictures it bakes. Any USD reader can draw these. One example is the Earth in the Moon's sky. Its continents, deserts, ice and cloud come from the game's own noise shader, ported to numpy and baked once into a longitude-latitude picture. The place's own sun lights the ball, so its night side falls dark. The sun's disc hangs along each place's sun as wide as the game draws it. The game's lamp, screen and grow-light materials are library surfaces.

Gap: the scene in a game engine. The same stage loaded in a game engine, with its collision and surfaces, and the mass and friction of each object written into it. It waits on the game engine adapter and on the framework recording mass and friction.

2.8 Review tools

Each place ends with the creator's review. The review tools that SCORE supplies are pages that show each stage's output side by side, before-and-after shots from the same cameras, walkthroughs inside the engine, and the result of every check. An optional world step can turn the concept into a whole room that can be walked through, so that walls and objects the concept does not show can be seen; nothing it produces is shipped.

A review page is a static folder built only from the files the stages wrote, with the assembled scene

rendered by Blender from the place's OpenUSD stage, on rented cards, from the cameras of its scene record and along a walk through it (Figure 8). Each view is laid beside the game's own shot from the same spot, and a view that is much darker than the game's fails a brightness check, which is how lamps whose light fell off too fast were found. A place with people shows each of them close and the crowd wide, and its walk is drawn at consecutive moments of the stage's time, so that the people move.

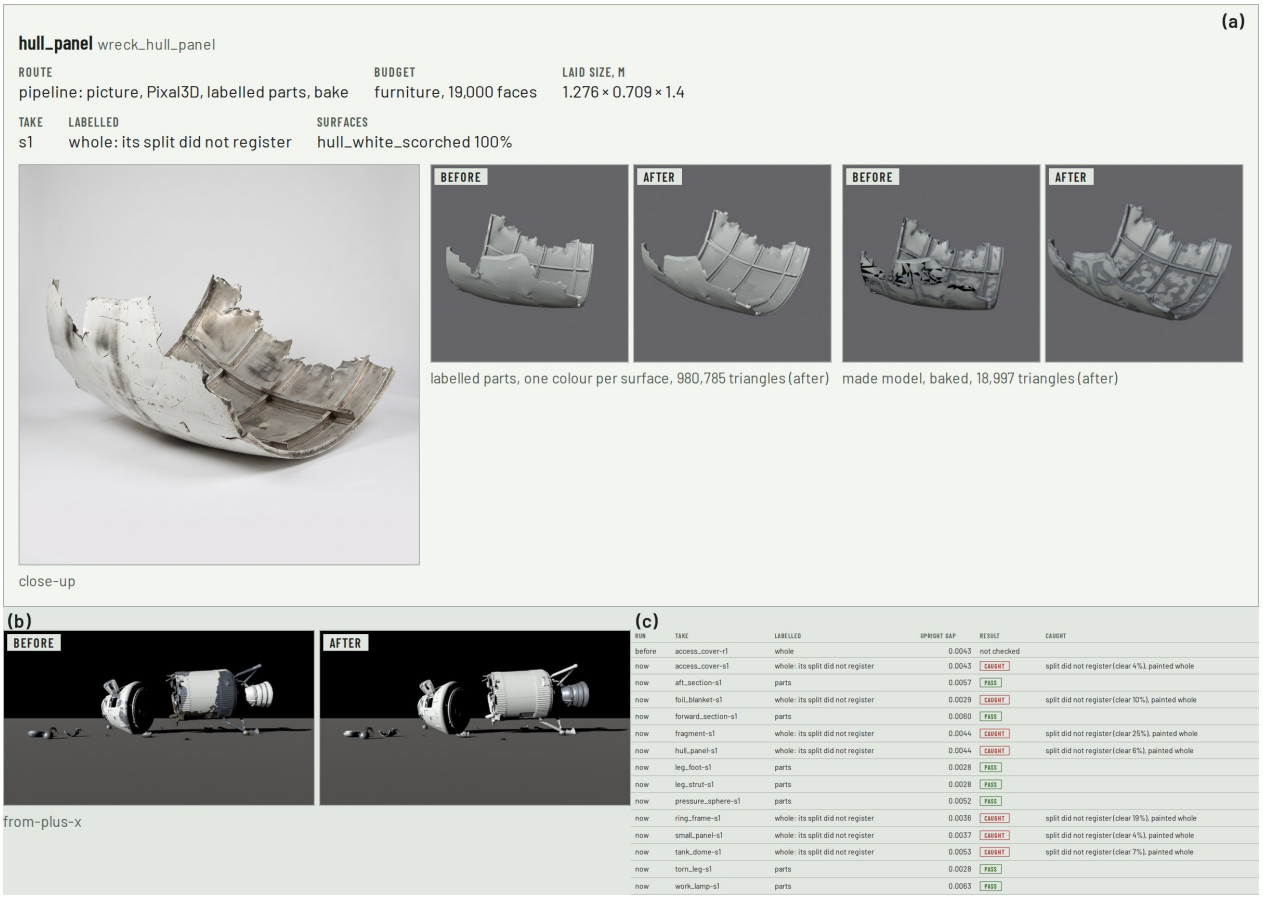


Figure 8: The wreck's review page, from two runs of the route over the same place. (a) One model: its close-up, its labelled parts (one colour per library surface) and its baked model, each from the first run and the rerun, drawn from the same camera. (b) The assembled scene from one fixed camera, before and after the rerun. (c) The parts check for each take, with what it caught in that rerun of 8 October: the split of seven takes did not register against their pictures, so each was painted whole.

3 Results across worlds

We build four worlds with the same stages: the game world of 2099 (a Moon base and its surroundings, a camp on Mars, and a prologue on Earth), a stylised underwater world, a simulation world for robots, and a third-person fantasy world.

Gap: Table 1, results per world and stage. For each world, this table will give the number of places, the wall-clock time, the cost in GPU time and in picture calls, the number of models built in code and generated, and the faults caught before spending at each stage against those found afterwards. All 17 places of world 1 are built and their records are in Appendix B, but they become results only when the creator has accepted each stage of each place. He accepted the hub (Table C1), reviewed the old station and the wreck, which have changed since, and found a placeholder in a demo of the world; on 10 October, 3 of the 18 places the completion gate judged passed it. Worlds 2 to 4 have not been started.

Gap: figure of one place per world. This figure will show one representative place from each world beside its concept, rendered from the concept's own camera. It waits on the same worlds; every place will be shown in Appendix A.

4 Evaluation

Gap: LEGO-Bench. This section will report SCORE's scores on LEGO-Bench [15], a benchmark of scenes rebuilt from pictures. It waits until the benchmark's code is released and until SCORE's scene export and its Blender adapter are built.

4.1 With and without the agent tools

The tools that check an agent's work (the completion gate, the resting triage with its drop test, the pick lock and the render-and-compare, Section 2) were tested on three places of world 1: a room (the workshop, 814 objects), an outdoor place (the street, 1,198 objects) and a dense room (the hangar, 1,657 objects). Each arm started from the same exported stage and the same inventory, and the same agent (Claude Opus 5.5 in Claude Code, headless) got the same prompt: make the place as complete as it can in about 50 minutes, through the place's layout data only. One arm had the framework as it was before these tools; the other had them. Both arms were measured afterwards with the same instruments (Table 2). Each arm ran once, so the table shows what happened, not an average.

Table 2. One session per arm and place. Faults are the resting triage's real faults left on the final stage, measured against the arm's own data; placeholders by the first placeholder check, which judged only plain primitives and so could not see a thing a code builder assembled from parts, such as the hangar's ship (Section 2); rows are inventory rows with no object and no written reason. Agent cost is the session's model cost in dollars; cloud is what its rented machines cost.

Place, arm	Real faults (start → end)	Placeholders	Rows missing	Minutes	Agent cost	Output tokens	Cloud	Exports	Gate said unknown
Workshop, without	18 → 13	0 → 0	1 → 0	36	$2.26	30.4k	€0.85	7	not used
Workshop, with	18 → 0	0 → 0	1 → 0	26	$1.42	14.3k	€0.85	2	2 of 4 calls
Street, without	11 → 0	0 → 0	0 → 0	38	$2.40	30.0k	€0.32	9	not used

Place, arm	Real faults (start → end)	Placeholders	Rows missing	Minutes	Agent cost	Output tokens	Cloud	Exports	Gate said unknown
Street, with	12 → 0	0 → 0	0 → 0	39	$1.21	10.4k	€0.76	1	0 of 5 calls
Hangar, without	14 → 4	5 → 0	0 → 0	47	$2.01	26.6k	€0.00	7	not used
Hangar, with	14 → 9	5 → 0	0 → 0	32	$2.34	29.1k	€0.64	5	3 of 6 calls

The tools made building cheaper more often than they made the scene better. On the workshop and the street, the arm with the tools used about half the agent's output and cost, and exported the stage once or twice instead of seven to nine times, because the triage named the few real faults among hundreds of alarms. On the workshop it left no real fault, but 13 of its 18 were cleared by writing what holds a piece up into the inventory (the tools on the tool board hang from it; the arm monitors are clamped to their bench), which the creator still has to confirm; the other five were moved off the wall they stood in. On the hangar the arm without the tools left fewer faults: the arm with them spent its time replacing five placeholder slabs with the kit's own wall panels and ribs and trying the drop test, which failed on a room with no ground mesh (fixed after the run), and the nine faults it left are models with stray parts below their feet that no pose can fix. In no arm did the completion gate's hook have to refuse a claim, because no agent claimed a place was complete; both arms reported what was left. Every unknown answer of the gate was a check not yet run on the stage as it then was, which is what unknown means; the agents ran the checks and went on. Concept fit and the per-part surface check are not in the table, because their tools are not built yet, and no row carries an owner's pick, so the pick lock was never tested by an agent.

Gap: ablations. This section will compare the framework with and without the optional world step, and with the checks before spending switched off, on the same places (the agent tools are compared above). It will measure how much of the concept is covered, how dense the built place is, the cost, the faults found after spending, and the creator's review. It waits until every stage runs from the framework repository.

4.2 Style coherence

Gap: style coherence. This section will measure whether a place keeps one style. A measure is built: from the review pages' renders of each place's generated models, the fit of their coloured pixels to the place's palette, the spread of their lightness, how blotchy their surfaces are, and DINOv2's likeness [16] within a place against across places with the shape subtracted. Its readings on world 1 are kept in Appendix B as a record. It is not yet a result, because it has not been checked against the creator's reviews; it waits on the surface stage's reviewed before and after, and it holds once it ranks the models the creator faulted below those he accepted.

Gap: robots in the simulation world. This section will show a rigged robot loaded together with the simulation world in a robotics simulator, with masses, friction and joints written into the scene, and with motion from the framework's motion models. It waits on world 3.

5 Limitations

- **One creator, one world so far, and that world not yet accepted.** Every verdict on the results is one creator's, and only the first world has been built. Its places do not yet pass their own completion gate: on 10 October, 3 of 18 passed, and the creator has accepted one place, the hub.
- **Code builders do not scale well.** They give clean objects, but a coding agent has to write one for every kind of object.
- **Places come out sparser than their concepts.** The concept-density check catches this but does not fix it.
- **One stage still depends on a closed, paid picture model.** Nano Banana Pro, a closed and paid service, draws the concepts and every close-up that the open model's pictures fail. On the last 17 close-ups of world 1, none of the open model's pictures passed the shape check, so the closed model drew every close-up that was accepted. Its daily limits set the pace of world 1, and it can change or be retired, which limits reproducibility and leaves a public framework depending on a closed service for one stage. A cheaper model of the same family did not replace it (Appendix E).
- **Parts do not follow finishes, and shadow reads as paint.** PartCrafter's parts follow shape, not paint, so a finish that does not have its own part, such as a gold foil patch on a white hull or a brown seat on a grey frame, is still lost when the part is painted with one surface, and the judge that names each part's surface reads a region the close-up shows in shadow as dark paint. On world 1 the painting is therefore not yet consistent beyond the hub; a split seeded by the close-up's regions (GeoSAM2 [46]) owns more finishes in a comparison on 22 models, but is not yet part of the route or shown on a repainted model.
- **Objects are physical but not interactive.** Seats, terminals and other usable objects do not work in the output.
- **Characters' clothes are rejected, and animals' motion is rough.** The creator rejected the Newton drapes that every crew and cast body wears, as bloated at the chest and sleeves, and they fail the silhouette check against Warp's drapes; of the open routes tried since, Newton with settings per look passes the work suits but no space suit. The drapes are a trial until one route passes that check on every look and then the creator's eye. A second way to make a person, Pixal3D's mesh with SOMA-X's skeleton fitted inside it, is also a trial: its first person still fails the check that the clothes stay tucked in. The animals' clips are made in code: the dog's paws twist, its sit reads as a crouch and its feet slide by up to 5.4 cm.

6 Future work

- Build the three further worlds: a stylised underwater world, a simulation world for robots, and a third-person fantasy world.
- Add a game engine adapter for the OpenUSD scene, and write each object's mass and friction into it.
- Make assets interactable, which robotics in particular needs.
- Finish the animals and the drapes: fins that move, animal motion retargeted from a captured set whose licence has been checked, and a drape route that passes the silhouette check on every look and the creator's review.
- Make the surface stage hold on world 1: parts split along finishes, read from close-ups without shadow, and each repainted model shown before and after to the creator.
- Make the open picture model's close-ups pass the shape check more often, so that the closed

model is needed less.
- Wrap open solvers as stages for worlds at other scales, starting with orbital dynamics through REBOUND [47].
- Evaluate SCORE on LEGO-Bench [15].

7 Conclusion

SCORE builds complete, owned and responsive worlds from a creator's choices, by joining open vision foundation models, procedural tools and coding agents into fixed stages whose outputs are kept. On its first world, the parts check, one model per real-world object and the shared library of rule-based surfaces made one room, the hub, consistent enough for its creator to accept it, and the deterministic checks before every stage that costs money caught faults before money was spent. The rest of that world does not yet pass its own completion gate. Whether the framework holds across a whole world, and across different kinds of worlds, is what the first world's accepted places, the remaining worlds and the benchmark will show.

References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[4] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[6] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[9] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[10] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[11] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[12] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[13] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[14] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[15] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[16] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[17] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[18] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[19] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[20] L. Yang, B. Kang, Z. Huang, *et al.*, "Depth Anything V2," *arXiv preprint arXiv:2406.09414*, 2024.

[21] Y. Lin *et al.*, "PartCrafter: Structured 3D mesh generation via compositional latent diffusion transformers," in *Advances in neural information processing systems (NeurIPS)*, 2025.

[22] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[23] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.

[24] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[25] Y. Li *et al.*, "Chorus: Multi-teacher pretraining for holistic 3D Gaussian scene encoding," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2026.

[26] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[27] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[28] B. Lei, Y. Li, X. Liu, *et al.*, "Hunyuan3D Studio: End-to-end AI pipeline for game-ready 3D asset generation," *arXiv preprint arXiv:2509.12815*, 2025.

[29] Qwen Team, "Qwen-Image-Edit-2511 model card." Hugging Face, 2025. Available: https://huggingface.co/Qwen/Qwen-Image-Edit-2511

[30] Qwen Team, "Qwen3.8-27B model card." Hugging Face, 2026. Available: https://huggingface.co/Qwen/Qwen3.8-27B

[31] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[32] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[33] H.-T. Tsui, Y.-R. Tuan, X. Ma, N. Ugrinovic, T. Shiratori, and K. Kitani, "Point2Part: Unified 3D partitioning from point prompts," *arXiv preprint arXiv:2609.38180*, 2026.

[34] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[35] H. I. I. Tam, H. I. D. Pun, A. T. Wang, A. X. Chang, and M. Savva, "SceneEval: Evaluating semantic coherence in text-conditioned 3D indoor scene synthesis," *arXiv preprint arXiv:2503.14756*, 2025.

[36] H. Xia *et al.*, "SAGE: Scalable agentic 3D scene generation for embodied AI," *arXiv preprint arXiv:2602.10116*, 2026.

[37] Black Forest Labs, "FLUX.2 [klein] 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-4B

[38] A. Qammaz, "SAM3DBody-cpp: A standalone C++ inference engine for SAM 3D Body." GitHub, 2026. Available: https://github.com/AmmarkoV/SAM3DBody-cpp

[39] M. Korosteleva and O. Sorkine-Hornung, "GarmentCode: Programming parametric sewing patterns," *ACM Transactions on Graphics*, vol. 42, no. 6, p. 197, 2023.

[40] The Newton Developers, "Newton: A GPU-accelerated physics simulation engine." 2026. Available: https://github.com/newton-physics/newton

[41] M. Macklin, "Warp: A high-performance Python framework for GPU simulation and graphics." NVIDIA, 2022. Available: https://github.com/NVIDIA/warp

[42] C. Ye *et al.*, "Hi3DGen: High-fidelity 3D geometry generation from images via normal bridging," in *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*, 2025.

[43] Black Forest Labs, "FLUX.2 [klein] Base 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-base-4B

[44] thedeoxen, "refcontrol: Reference-depth LoRA for FLUX.2 [klein] 4B." Hugging Face, 2026. Available: https://huggingface.co/thedeoxen/refcontrol-FLUX.2-klein-4B-reference-depth-lora

[45] J.-P. Zhang, C.-F. Pu, M.-H. Guo, Y.-P. Cao, and S.-M. Hu, "One model to rig them all: Diverse skeleton rigging with UniRig," *ACM Transactions on Graphics (SIGGRAPH)*, 2025.

[46] K. Deng *et al.*, "GeoSAM2: Unleashing the power of SAM2 for 3D part segmentation," *arXiv preprint arXiv:2508.14036*, 2025.

[47] H. Rein and S.-F. Liu, "REBOUND: An open-source multi-purpose N-body code for collisional dynamics," *Astronomy & Astrophysics*, vol. 537, p. A128, 2012.

[48] Blender Foundation, "Blender." 2026. Available: https://www.blender.org/

[49] O. Siméoni, H. V. Vo, M. Seitzer, *et al.*, "DINOv3," *arXiv preprint arXiv:2508.10104*, 2025.

[50] L. Li *et al.*, "SegviGen: Repurposing 3D generative model for part segmentation," *arXiv preprint arXiv:2603.16869*, 2026.

[51] World Labs, "Terms of Service." 2026. Available: https://docs.worldlabs.ai/terms-of-service

[52] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, "LLM2Vec: Large language models are secretly powerful text encoders," in *Conference on language modeling (COLM)*, 2024.

[53] A. Grattafiori, A. Dubey, A. Jauhri, *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[54] R. Abdrashitov, K. Raichstat, J. Monsen, and D. Hill, "Robust skin weights transfer via weight inpainting," in *SIGGRAPH asia 2023 technical communications*, 2023.

[55] Anthropic, "Claude Code." 2026. Available: https://www.anthropic.com/claude-code

Appendix

A Per-world demos

A.1 World 1: the game world of 2099

Gap: world 1's demo. One frame and a walk-through of every place of world 1, rendered from the stage the creator accepted for it. A first render of 9 October exists, one frame and a silent walk-through of each of the 17 places from the stages of that day (5,292 frames on 20 rented L4 cards, 78 minutes, €15.82); those stages have changed since, the creator found a placeholder in a later demo of the world, and none of its places is accepted beyond the hub, so it is not shown. It waits on the creator's accept of each place.

Figure A1 follows two objects through the stages, from the concept to the place.

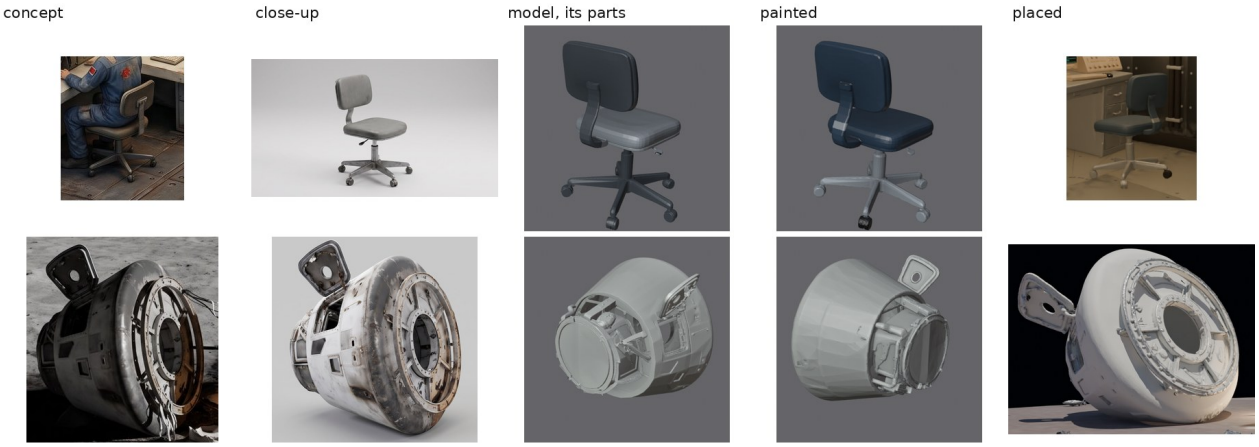


Figure 9: Two objects through the stages: the lab's desk chair (top) and the wreck's forward section (bottom). From left: the concept the creator chose, the close-up drawn from it, the generated model with its parts, the model painted from the surface library, and the model in its place on the stage. The forward section shows a known fault: its scorched paint has no part of its own and is lost in painting (Section 5).

Gap: worlds 2 to 4, and a public link to the videos. This appendix will show every place of the other three worlds in the same way. The videos of world 1 are not yet published at a public address.

B Detailed per-stage results

Table B1 gives the records of world 1's 17 places as the build logs and the cloud ledger hold them. They are records, not yet results: the creator has accepted only the hub, the old station and the wreck have changed since he reviewed them, no other place has been reviewed, and worlds 2 to 4 have not been started. The cloud costs are lower bounds. The ledger attributes a batch to a place by the work folders and take names in it; review renders (€24.24), the shape-check judge (€22.81), the game's shared test machines (€32.63), settling (€4.43), the open picture models (€10.27) and the repainting and spreading work (€8.44) could not be attributed to one place. No time or cost is recorded for each stage of a place separately, only for each batch, so the table gives none. The resting check is the one on each place's review page of 8 October, before the night's complete stages were exported; it counts every piece it fails, including many deck plates that it finds sunk by 4 cm,

which have not been judged as real faults or not.

Table B1. World 1, per place. Models are those built in code and those generated. Cloud is the ledger's rented machine time attributed to the place, in euros; pictures are the picture model's calls in dollars, from the build logs. Faults are those the checks caught before money was spent and those found after.

Place	Rows	Pieces	Models, code / generated	Cloud	Pictures	Faults before / after	Resting
Hub	74	460	88 / 12	€3.65	$1.07	not counted	405 of 460
Lab	47	412	54 / 22	€8.36	$4.15	13 / 12	373 of 412
Greenhouse	59	1,248	146 / 3	€3.07	$2.13	7 / 3	1,154 of 1,248
Workshop	54	814	97 / 8	€2.18	$3.35	3 / 9	671 of 814
Airlock	33	330	73 / 1	€2.53	$0.13	1 / 5	296 of 330
Walkway tube	13	39	23 / 0	€0.81	$0	0 / 2	38 of 39
Habitat	53	637	100 / 2	€5.70	$1.34	1 / 4	543 of 637
Garage	41	602	75 / 19	€4.29	$8.71 with the hangar	3 / not counted, with the hangar	493 of 602
Hangar	37	1,686	61 / 13	€4.22	with the garage	with the garage	865 of 1,686
Wreck	14	22	3 / 13	€4.88	$2.68	2 / 1	22 of 22
Old station	17	28	10 / 14	€4.93	$3.22	5 / 2	29 of 29
Mars camp	33	218	51 / 5	€3.47	$1.21	4 / 8	127 of 218
Launch view	4	67	4 / 1	€0.33	$0.40	1 / 2	8 of 67
Square	40	4,135	74 / 2	€1.64	$1.07	3 / 4	3,712 of 4,135
Street	33	1,198	45 / 2	€0.98	$1.74	7 / 6	1,179 of 1,198
Stairwell	21	118	42 / 1	€1.23	$2.41 with the flat	2 / 6	117 of 118
Flat	38	52	35 / 9	€2.44	with the stairwell	5 / 6	50 of 52
World 1	611	12,066	981 / 127	€54.71			10,082 of 12,066

B.1 Style readings

Table B2 gives the style readings for every place of world 1 with more than two generated models, as a record of the measure: they are not checked against the creator's reviews, so they are not a result (Section 4). The lightness spread is the standard deviation of the models' median lightness (L*); over all 140 models it is 15.0. Marks is the median over the models of the 75th percentile of the L* spread within tiles of 8 by 8 pixels on the model. Likeness is the mean DINOv2 class-token cosine of pairs within the place and across places, for the renders and for their silhouettes, and the gap is the renders' difference less the silhouettes'. The airlock, the launch view and the stairwell have one generated model each and the tube none; the habitat, the square and the street have two, too few to read.

Table B2. Style readings of world 1's generated models, per place.

Place	Models	Lightness spread	Marks	Likeness within / across	Silhouettes within / across	Gap
Camp	5	8.3	4.55	0.345 / 0.136	0.378 / 0.164	−0.005
Flat	9	16.7	4.83	0.186 / 0.122	0.243 / 0.176	−0.003
Garage	19	12.6	2.80	0.185 / 0.159	0.199 / 0.177	0.004
Greenhouse	3	4.5	6.12	0.071 / 0.140	0.140 / 0.174	−0.035
Hangar	13	9.5	2.99	0.185 / 0.151	0.231 / 0.174	−0.023
Hub	12	3.8	5.96	0.243 / 0.151	0.243 / 0.160	0.009
Lab	22	14.4	1.96	0.203 / 0.162	0.195 / 0.174	0.020
Old station	24	12.7	8.96	0.221 / 0.156	0.217 / 0.175	0.023
Workshop	8	3.8	7.28	0.402 / 0.158	0.400 / 0.178	0.022
Wreck	16	12.8	3.98	0.241 / 0.161	0.257 / 0.175	−0.002

For the comparison with the pictures' own colours, each generated model of the garage, the hangar and the lab was rendered a second time as the take it came from, with the colours the picture model gave it. Painted from the library, the median marks fell from 2.99 to 2.80 in the garage, from 3.53 to 2.99 in the hangar and from 2.79 to 1.96 in the lab. The lightness spread rose, from 7.9, 6.4 and 8.7 to 12.6, 9.5 and 14.4, because the library gives a place dark and light panels on purpose while the generator's colours stay in the middle tones. The DINOv2 gap read 0.007 painted against 0.000 with the picture's colours, a difference for which no interval was computed. The take is the generator's own mesh, at its own pose and density, while the made model is rebuilt from its parts, so the difference in marks mixes the paint with the geometry. The workshop's models are the hub's

tools, reused under the rule against duplicates; any two renders of one model, in two places or within one, are left out of the likeness, which would otherwise count one model as two that agree.

C Development history

The framework grew out of building the game 2099 between late September and early October 2026, with one owner directing coding agents. In its first form it joined four parts: rooms laid out in code, a scene plan from a world model that painted depth panoramas of those rooms, props from a picture-to-3D pipeline, and coding agents that fitted each room to its plan. Eight places were fitted out to their plans in a single day. The owner found that their layouts were close to the plans, but that their styles were mixed, with models from earlier rounds and with objects that the plans never showed. The cause lay in the workflow: every pass added things to a room, and no pass checked what had been placed against the plan. This led to the inventory as a strict list, with nothing made or placed outside it, and to deciding the look before any money is spent. The structure of each room then came from concept cutaways drawn from the owner's own reference pictures; for the hub he picked one cutaway out of twenty. Scale moved into a dimensioned plan drawn before the concept, after the first cutaway came out at doll's-house scale: about 13 m across, judged by the people drawn in it, against the 8 to 9 m that had been asked for.

C.1 The hub in six rounds

The mechanisms of Section 2 then came out of six rounds of work on one room, the central hub of the game 2099, on 6 and 7 October 2026. Each round answered the creator's verdict on the round before (Table C1). In the first two rounds, the creator chose between the old and the new method by comparing them side by side from the same cameras.

Table C1. The hub's six rounds. Cost is given as rented GPU time in euros and picture-model calls in dollars. Frame time is the mean frame time on an RTX 5080, in the night scene, at 1600×900.

Round	Change	Creator's verdict	Cost	Frame time
1, two walls, A/B	surface library, routing by shape, room checks	chose the new method ("Y looks much cleaner")	€0.88	7.81 ms vs 7.53
2, two walls, A/B	71-variant library, shared texture sets, roof check	chose the new method again	€1.70	8.05 ms vs 8.24
3, whole room	all stages end to end, 74 models	old models were still loaded; labels and floor fittings were wrong	€1.67	6.78 ms (old 9.22)
4, whole room	detail through the pipeline, picture colours kept	"pretty much all of this became messy"	€8.50, $5.09	9.6 ms

Round	Change	Creator's verdict	Cost	Frame time
5, eight pieces, A/B	shape only from the pipeline, library surfaces, parts check	new method preferred for the door, notice board and porthole; three pieces still wrong	€0.21	not run
6, whole room	round 5 everywhere, one model per object, straightness check	"this looks amazing, I have no negative feedback"	€3.48, $1.07	5.6 ms

Three findings shaped the method. First, deterministic checks catch what agents miss. In round one, the share of rays that leaked out through walls and roof fell from 1.53 percent of 5,817 rays to zero, and the colour noise within surfaces fell from a median of 31.1 to 3.2 percent. Only walking through the loaded room, however, showed that 26 old models were still in it, and that finding became the made-only check. Second, generated pieces that kept their pictures' colours made the room inconsistent, so surfaces have since come only from the library. Third, generated composite objects melted their parts together, and generated boxes leaned (one locker by 13 degrees). This led to one model per real-world object and to the straightness check (Figure 4b). By round six the room used 0.45 million triangles, against 1.96 million in round four, and 187 MB on disk, against 487 MB.

D Check details

- **Room checks** take about two seconds per room and run on the laid-out meshes. They cast rays to find light leaking out through walls and roof, find pieces placed in front of a real opening, find roof and wall fittings more than 3 cm off their surface or turned more than 5 degrees from it, and count pieces, triangles and lamps against the room's budget.
- **Door check:** every door must stand in a wall or partition that fully separates its two sides. The check tests this by walking a grid from one side to the other.
- **Concept-density check:** the concept picture is cut into crops, and every visible element must have an inventory row or a written reason for leaving it out.
- **Parts check:** in the code build, every part shown in the close-up picture must be at least 10 percent visible from the front, and no label may cover a part.
- **Model check:** every model must be a closed solid with walls at least 3 mm thick. A generated box fails if one of its sides tilts more than 2 degrees or is warped. The step that makes a generated model solid stops itself if the model's bounding box grows more than 3 percent.
- **Palette sweep and storage budget:** the colours of every model are compared with the place's library palette, and each room may use at most 250 MB on disk and 640 MB of textures on the graphics card.
- **Scene check and made-only check:** in the engine, these checks find anything that floats, sinks or overlaps, and any visible mesh that the framework did not make. A made-only finding can never be waived.
- **Close-up shape check:** the outline must lie within a set proportion of the inventory row's

box, and each of the judge's seven questions must pass by a majority of three sampled answers. A close-up passes only when both halves pass.

- **Settling and resting check:** on the OpenUSD stage, loose objects are dropped with their own triangles onto the ground and the fixed objects. An object that would turn more than 15 degrees or drift more than 30 cm keeps its laid pose and is marked as not resting; small debris, with no side over 1.4 m, may turn freely and drift up to 1 m. The resting check then fails anything floating, tipping, sunk below its contact points, lying inside another piece or marked as not resting. Hung objects are exempt by their inventory anchor, and fixed objects are not judged for tipping.
- **Resting triage:** each object's support comes from its inventory row: a glowing part or screen is judged with its host; a fixed row stands as laid but may not float unless joined to another piece; a kit's shell pieces stand in the kit's frame; a hanging row and a wall or ceiling row need one contact with anything; a floor row and a row standing on another need four points of their footing (the points within 10 cm of their lowest) within 3 cm of a surface under them, with their weight within 5 cm of the area those points span. An overlap deeper than 3 cm counts only if the two still meet after the loose one moves 5 mm away. Cables may sink 6 cm, cloths 8 cm and ground debris 30 cm. From a drop run, SAGE's stability rule of Section 2 marks a real fault.
- **Completion gate:** the requirements given in Section 2 under the canonical scene, each with its result tied to a fingerprint of the stage's files and inputs; a result made on another stage counts as unknown.
- **Pick lock and render-and-compare:** a picked row's objects must show the model with the picked hash, or the row must carry the creator's written replacement. Each object with a close-up is rendered from eight directions 20 degrees up, and the direction with the highest DINOv2 likeness stands for the close-up's camera; the outline score is the overlap of the two silhouettes, each cut to its box.
- **Brightness check:** each view of a place's stage on its review page is shown beside the game's shot from the same spot, and a view whose mean brightness is under half the game's fails.

E Models and licences

Every model in the framework must allow commercial use of its output (Table E1). Candidate models were dropped under this rule; among them were NVIDIA's Lyra 2.0, whose weights are licensed for internal research only, and Hunyuan3D 2.1, whose licence does not apply in the European Union. We read each licence for its clauses on commercial use. Four of the models read pictures through Meta's DINO family of models, each through its own copy. What passes from one model to the next is always pictures, masks, depth maps or meshes, never one model's internal features. The characters' licence check found that GarmentCode's fork of NVIDIA Warp, which drapes its patterns, allows non-commercial research only, so Newton's cloth solver on upstream Warp, both Apache-2.0, replaced it; Blender's cloth simulation [48] is kept as a second route. The one open doubt among the characters' models is the face LoRA, whose training data is not disclosed.

Table E1. Models in the framework.

Model	Role	Licence
Nano Banana Pro [31]	concepts, close-ups the open model fails, ground skins	paid API
Qwen-Image-Edit-2511 [29]	close-ups, first	Apache-2.0

Model	Role	Licence
Qwen3.8-27B [30]	the close-ups' shape check; each part's material	Apache-2.0
Pixal3D [17]	picture to 3D	MIT; DINOv3 licence [49] for its encoder
PartCrafter [21]	parts	MIT; its non-commercial background remover not used
SegviGen [50]	parts of our own model (trial)	MIT, on TRELLIS.2 (MIT); DINOv3 licence for its encoder; its non-commercial background remover and nvdiffrast not used
GeoSAM2 [46]	parts of our own model, seeded by the close-up's regions (trial)	Apache-2.0
Depth Anything V2 Small [20]	ground relief	Apache-2.0
ProcFunc, Infinigen shaders	surfaces	BSD-3-Clause
MOSS-SoundEffect v2.0, CLAP	sound	Apache-2.0
World Labs Marble [51]	optional world step	paid; outputs owned by paid users
FLUX.2 klein 4B [37]	earlier prop pictures; the characters' A-pose pictures from words and their hair's clay pictures	Apache-2.0
MoGe-2 [19], SAM 3 [18]	measuring and finding objects in pictures	MIT; SAM Licence
Kimodo [23]	the characters' motion from sentences	Apache-2.0 code; NVIDIA Open Model License for the Kimodo-SOMA-RP-v1.1 weights, commercial use allowed
LLM2Vec [52] on Meta-Llama-3-8B-Instruct [53]	Kimodo's text encoder, used only while clips are made; nothing from it ships	Llama 3 Community License: commercial use allowed, the 700 million monthly users clause does not apply, "Built with Meta Llama 3" shown, no exclusion in the European Union for the text-only 8B model
SAM 3D Body [22], SAM3DBody-cpp [38]	body shape from a picture	SAM Licence for the weights, SAM3DBody-cpp's converted ONNX and GGUF weights included whatever their tag says; MIT for SAM3DBody-cpp's code
SOMA-X and MHR	the characters' bodies	Apache-2.0

Model	Role	Licence
GarmentCode [39], Newton [40] on Warp [41], Blender's cloth [48]	the clothes' sewing patterns and their drape	MIT; Newton and Warp Apache-2.0; Blender is GPL and its output is ours; GarmentCode's Warp fork (non-commercial research only) not used
Hi3DGen [42]	the characters' hair	MIT code; MIT and Apache-2.0 weights
FLUX.2 klein base 4B [43] with the refcontrol reference-depth LoRA [44]	the characters' faces	Apache-2.0, both; the LoRA's training data is not disclosed
UniRig [45]	the skeleton and skin of a quadruped	MIT code and weights
Robust Skin Weights Transfer [54]	the SOMA-X skeleton's skin weights moved onto a Pixal3D person (the prop route of the character maker, a trial that fails its checks and has made no person of world 1)	MIT reference code, ported with trimesh and SciPy; libigl, whose bindings carry GPL-3.0, not used
MakeHuman eyes	the characters' eyes	CC0

On our tier, Nano Banana Pro allows 250 pictures a day and 20 a minute; the cheaper Nano Banana (gemini-2.5-flash-image) allows 2,000 a day and 500 a minute. We tested whether the cheaper model could draw the close-ups instead, on the same 10 lab objects with the same prompts and inputs (Table E2). It refused the standard prompt for 6 of the 10 objects and drew those only from the crop alone, and its pictures were about one megapixel with the object small in the frame. Close-ups therefore stay on Nano Banana Pro.

Table E2. Ten lab close-ups drawn by Nano Banana instead of Nano Banana Pro.

Result	Objects
good	2
usable	3
marginal	1
wrong shape (microscope, chair, sample tray)	3
refused (glovebox)	1

The close-ups' shape check was tuned on 40 close-ups that four open picture models drew of the lab's ten objects, each scored by hand as good, usable, marginal, wrong or failed, and on Nano Banana Pro's ten pictures of the same objects. With the majority of three sampled answers, the check agreed with the hand score on 29 of the 40, passed all ten of the closed model's pictures and two marginal ones, passed none of the pictures scored wrong or failed, and failed nine that were good or usable, each of which then cost a picture from the closed model. The judge's single answer at temperature zero was rejected: asked the same 51 questions twice in one batch, it gave a different verdict on 7 of them. On the lab's 22 generated rows, an earlier version of the check accepted 8 of

the open model's close-ups and sent 14 to the closed model; 7 of the closed model's first pictures failed for drawing the room or dimension lines around the object, and all 7 passed when redrawn from the crop alone. On the 17 rows that world 1 still needed afterwards, from three places, none of the open model's pictures passed; the closed model's pictures were accepted for 10 rows, and 7 rows were left for the creator to look at.

Table E3. The close-up stage on world 1. Cloud is rented card time in euros; the closed model's pictures are in dollars.

Run	Close-ups	Open model accepted	Closed model accepted	Left for the creator	Cloud	Closed model
Lab, 22 rows (earlier check: one answer)	22	8	14	0	€4.12	$2.81
World 1's remaining rows (three places)	17	0	10	7	€3.06	$4.02

The picture-to-3D step, the part splitting, the surface bakes, the sound generation, all checks and the assembly run automatically. Coding agents (Claude Code [55]) write the plans, the inventories and every code builder; in the hub's final round, 86 of its 98 models were built in code. Some steps were still done by hand: some generated objects were turned to face the right way by eye, and a few labels were placed by hand before a rule took over that job. The concepts and the verdicts are the creator's own choices.

F Surfaces, sound and light

The surface library holds families of variants: painted, steel, aluminium, rubber, plastic, cable, fabric, composite, glass, screen, light and print. After the hub's second round it held 71 variants made from 14 ProcFunc recipes. Each variant is a recipe with settings, takes its colours only from the palette, and is baked into base-colour, metal-roughness and normal maps at a texture density that depends on how close the player can get. Wear has three levels and comes from causes: edges, and kicks within 32 cm of the floor that a piece stands on. Every sound starts from a short written brief about its object. Four takes are generated for each sound, scored by how well CLAP matches them to their prompt minus any measured faults (clipping, unsteady loops or bad joins), and levelled to one loudness per kind of sound under a true peak of -3 dBTP. A creator may swap any take. For the game 2099, all 61 sounds were generated as 244 takes in one batch for €1.61, and the creator chose to keep earlier recordings for three kinds of sound: breathing, a sliding door, and footsteps on steel.

Light adds up, so it is planned as layers. For each kind of module, the bounce light of each group of lamps, and of a few sun directions through the windows, will be baked on rented machines as a separate layer, and the engine will mix these layers live according to each lamp's brightness. Outdoor light and the light on moving objects stay live. This part is not built yet.

G Cost and infrastructure

Every heavy step runs on rented machines that are deleted after each batch: NVIDIA graphics cards for picture-to-3D, part splitting and baking, and processor-only machines when no cards are in stock. The engine's tests, its screenshots and its frame-time benchmark also run on rented machines. A full test run took 542 s there, against 596 s on the local computer, and frame times measured on an L4 card are converted to the local RTX 5080 by a measured factor of 3.47. A picture costs about $0.134 at the resolution we use. The picture model's limits on speed are described under Limitations.

A batch asks for a class of machine, such as one card with 24 GB, and takes whatever the provider has in stock across three zones and three card types, because on 8 October the cheapest type was out of stock in one zone for most of the day. Table G1 compares the same jobs on two card types. An H100 runs ten Pixal3D takes at once in 31 GB of its 80 GB, and makes nearly three times as many models an hour as an L4. It has no ray-tracing cores, so it bakes 2.6 times slower than an L4. Pixal3D therefore takes 80 GB cards first, and a bake takes an H100 only when no other card has been free for five minutes. The L40S was out of stock on both attempts, and the provider's P100 machines started but their driver did not see the card.

Table G1. The same jobs on two card types, 8 October 2026. Wait is the time from the order until the machine answers; machine time is the whole rental, setup included.

Card	Job	At once	Wait	Machine time	Cost	Per take
L4, 24 GB	Pixal3D, 3 close-ups	3	0.4 min	28.0 min	€0.38	€0.13, 6.4 an hour
H100, 80 GB	Pixal3D, the same 3	3	0.6 min	13.9 min	€0.67	€0.22, 12.9 an hour
H100, 80 GB	Pixal3D, 10 close-ups	10	0.6 min	33.1 min	€1.62	€0.16, 18.1 an hour
L4, 24 GB	bake: 60 surfaces at 3 wear levels	1	0.4 min	3.8 min (bake 138 s)	€0.05	
H100, 80 GB	the same bake	1	0.6 min	7.5 min (bake 360 s)	€0.38	

Since 8 October, every kind of batch is spread over many machines at once: each machine is set up once and takes the next job from one queue as it finishes one, and a batch rents enough machines that each works about as long as its setup takes. Table G2 compares the same batches on one machine and spread. A spread batch ended 1.2 to 2.7 times sooner and cost 4 to 63 percent more, because every added machine spends part of its time setting up.

Table G2. The same batches on one machine and spread over several, 8 October 2026.

Batch	Machines	Time	Cost	On one machine
Blender, 6 jobs	3 L4	7.0 min	€0.28	14.3 min, €0.20

Batch	Machines	Time	Cost	On one machine
Library bake, 22 jobs	4 L4	10 min	€0.43	27 min, €0.35
Pictures (FLUX.2 klein), 120	3 L4	5 min	€0.20	10 min, €0.14
Pictures without light, 32	2 L4	4 min	€0.10	6 min, €0.08
PartCrafter, 12 runs	2 L4	19.3 min	€0.49	35.2 min, €0.47
Shape-check judge, 184 questions	2 H100	16 min	€1.48	19 min, €0.91
Qwen-Image-Edit, 16 close-ups	3 H100	11 min	€1.53	26 min, €1.24

Table G3 sums the cloud ledger, in which every runner records each machine it rents, by type of machine, from the first batch on 29 September to the end of 8 October 2026. It holds the game 2099's own test and benchmark machines as well as the framework's batches. €35.68 has no type: the machines of the earliest batches were recorded without one (€34.69), and some rows hold €0.99 more than the machines they list.

Table G3. Rented machines by type, 29 September to 8 October 2026.

Machine	Rentals	Machine time	Cost
L4, 24 GB	793	331.1 h	€266.19
H100 PCIe, 80 GB	29	7.4 h	€21.69
two H100 SXM, 80 GB each	14	1.5 h	€11.03
32-core processor, 128 GB	26	5.4 h	€9.85
two L4, 24 GB each	3	3.4 h	€5.38
16-core processor, 64 GB	13	2.3 h	€4.81
32-core processor, 64 GB	5	0.9 h	€4.26
L40S, 48 GB	6	2.7 h	€4.07
P100, 16 GB	2	0.1 h	€2.44
16-core processor, 128 GB	2	1.6 h	€1.36
type not recorded			€35.68
total			€366.76

The character maker runs a person's whole chain on one card (Table G4). Its machine is set up once with six environments side by side, after the CUDA 12.6 toolkit that SAM3DBody-cpp needs: 11.0 minutes on an H100 and 21 to 25 minutes on an L4. On an H100, SAM3DBody-cpp's network takes 1.3 s and its whole call about 4 s once warm, 44 s cold. Against the Python SAM 3D Body on the same picture, its shape parameters correlate at 0.98, and its body at rest is 23 mm shorter (1.548 m against 1.571 m), with a mean gap of 17.6 mm between vertices and joints 19 mm apart on average and 25 mm at worst. The difference comes from the person box: SAM3DBody-cpp finds its own with YOLO, while the Python run was given the whole picture. Kimodo with its text encoder on

the same card writes a clip in 30 s once warm, and the first in 82 s, which includes downloading the Llama 3 encoder. Nev's 19 clips had been made already, so her rebuild wrote none. On her walk, the built character's skinning, with 27 joints and thinned weights as an engine plays it, keeps within 6.8 mm of the body model's own posed points on average and 52.6 mm at worst (4.9 mm on average off the hands), and its file is 5.05 MB. Her far body has 612 vertices and 1,220 triangles, her work suit 31,260 triangles and her space suit 44,846. The development machine on which the chain was built and Nev was rebuilt ran for 163 billed minutes and cost €7.79 in all. On an L4, the same chain made the player from the creator's drawing of him, on his own body 1.742 m tall, in 26.2 minutes, for about €0.35 of the card at €0.79 an hour. The face is the slow step on an L4: klein base 4B with the LoRA, 50 steps and two seeds, took 13.6 of those minutes against 2.7 on the H100. The L4 was set up in 24.8 minutes. A new person made from words alone, an engineer, first reached the build on a second L4 and failed at the arm stripe of her space suit, because the drape had left her upper arm bare; the drape now tries again when a sleeve does not cover the upper arm. The batch of both people cost €1.50 for 60 minutes on three L4 machines, one of which failed its setup after 25 minutes because Kimodo's C++ extension was built while the CUDA toolkit installed beside it; the toolkit now installs first. Made again on an H100, the engineer took 20.9 minutes: klein drew her picture in 0.27 minutes, the drapes took 6.65 and the review renders 5.96, because a fresh H100 first compiles Cycles' kernels. On a 16-core processor machine, Blender drapes the work suit in 1.5 minutes and the space suit in 0.6 minutes, with every seam within 3 cm and no cloth inside the body. Newton drapes a garment in 2 to 4 minutes on an L4 (Table G6). Draping the 16 crew and cast bodies again with Newton and building them cost €19.31 on L4 machines over three rounds; the first took 12 machines for 64 minutes and €7.38, and the other two mended the trousers' waist hold and the boots.

The animals' route rents cheaper cards. One Pixal3D run on an L40S made both animals' meshes in 11.2 minutes for €0.29, and UniRig rigs a model in about 70 s on an L4. The rig, the clips, the far levels and the review renders take 5.2 minutes of an L4 for €0.08. A processor machine costs 10 to 15 times more for the same job: the fish's own Blender run went to a 32-core processor machine when no card was free and cost €0.85. UsdSkel skins the animals to within 0.2 µm of their glTF skinning for the dog and 0.01 µm for the fish. The fish's levels have 40,000, 6,000 and 532 triangles, the dog's 39,999, 5,999 and 1,500.

Table G4. Characters made by the character maker, 9 October 2026. Minutes of card time per group of steps; a person's cost is the card time of the chain alone, without the setup; the fish and the dog shared one Pixal3D run on an L40S, split evenly here, and their costs are on cards for every step.

Character	Card	Picture	Body or mesh	Rig and clips	Clothes	Head, hair and face	Build and review	Total	Cost	Setup
Nev, from her picture	H100 PCIe	0 (her own)	0.13	0 (made already)	4.75	4.23	2.09	11.2 min	€0.54	not measured cleanly

Character	Card	Picture	Body or mesh	Rig and clips	Clothes	Head, hair and face	Build and review	Total	Cost	Setup
The engineer, from words	H100 PCIe	0.27	0.91	0 (made already)	6.65	5.21	7.84	20.9 min	€1.00	11.0 min
The player, from the creator's drawing	L4	0 (the creator's drawing)	0.34	0 (made already)	5.18	17.08	3.60	26.2 min	€0.35	24.8 min
Dog, from words	L4; L40S for the mesh	2 (4 pictures)	5.6	about 1.2 (UniRig)			5.2	about 14 min	about €0.40	about 1.5 min (UniRig)
Fish, from a close-up	L40S for the mesh; L4	0 (given)	5.6	in code			on an L4 as the dog's		about €0.32	

Since 9 October every stage that loads a model can also run from a pinned container image, one image per kind of job, built from files in the repository and pushed to a private registry chosen through the provider interface. The weights are not in the images: a job fetches each model once per node from our own object storage into a cache on the node, checked by its hash, and the first node to need a model seeds the storage from its pinned source. Each model's licence is recorded beside its image. Table G5 compares the time from a machine answering to the start of work under the old route, in which a fresh machine installed its software and downloaded its weights itself, with the image route on a new node. The old route was already fast for the kinds whose software comes as prebuilt packages, so the images gained little there and were slower for the largest weights, which come from our storage more slowly than from Hugging Face. They gained where the old route compiled or built: the part splitters, the character chain, and the first Cycles render on an H100, which compiled its kernels for 218 s because Blender ships no binary for that card and now reads them from a cache kept per card and driver. On a warm node an image's weights are ready in under a second. World 1 was built on the machine runners, which do not use the images; the image route and the Kubernetes path below are measured on their own, not yet used for a world.

Table G5. Cold start per kind of job, 9 October 2026: from the machine answering to the start of work. The old route installs and downloads on a fresh machine; the image route pulls the image and reads the weights from our storage on a new node.

Job	Card	Old route	Image route (pull + weights)
Pixal3D	L4	1.5 min	1.2 min (13 s + 58 s, 9.2 GB)
PartCrafter	L4	1.0 min	1.0 min (47 s + 14 s, 4.0 GB)
GeoSAM2 and SegviGen	H100	5.2 min	4.0 min (111 s + 128 s, 23 GB)
shape-check judge, Qwen3.8-27B	H100	1.0 to 1.5 min	3.5 min (62 s + 146 s, 31 GB)
Qwen-Image-Edit	H100	1.0 min	4.4 min (30 s + 231 s, 58 GB)
FLUX.2 klein	L4	0.8 min	1.8 min (37 s + 68 s)
SAM 3 cut-outs	L4	0.9 min	1.2 min (57 s + 13 s)
MOSS sound	L4	not measured	1.8 min (61 s + 44 s)
character chain (six environments)	L4	24.8 min	motion and body steps 2.7 min (72 s + 90 s)
first Cycles render	H100	218 s	0.3 s, kernels from the cache

The same images run on any Kubernetes cluster. A job becomes one Kubernetes Job, allowed on its kind's classes in the order of Table G1's rules, on node pools per class that scale from zero with the cluster's autoscaler; when no pool of those classes can get a node, the job is widened to the next class and keeps the ones it had, so it never settles for one card. A job writes its result last and is skipped when that result exists, so a run resumes after a lost node. On Scaleway's managed clusters, whose control plane is free, an L4 node went from the order to a running job in 249 to 326 s in five runs: the node was ready after about 140 s and NVIDIA's driver setup took about 2.5 minutes more, against a rented machine that answers in 0.5 to 1.5 minutes. One job alone therefore finishes later on a cold cluster (a PartCrafter split took 8.1 minutes against 4.6 and cost €0.20 against €0.07, with identical output), while a job on a warm node starts in about a second. When an L4 zone was out of stock the autoscaler moved to the next zone within a minute, and when processor machines were out of stock in both Paris zones for 20 minutes, a cluster spanning Warsaw took one there.

Table G6. Drapes of the same GarmentCode patterns on the same three bodies by three simulators, 9 October 2026, measured by tools/characters/maker/drape_measure.py: the trousers' hem above the floor, the mean bend between neighbouring cloth triangles over the whole garment (crumple) and between the hips and the lowest ribs (waist), and the height of the space suit's belt band that the torso cloth covers. Warp is GarmentCode's own fork, run as a reference only; Newton is the solver of the drapes the crew and cast wear, which the creator rejected and which fail the later silhouette check (Limitations): these measures did not see that fault.

Body and garment	Warp: hem, crumple, waist, belt	Blender's cloth	Newton
Nev, work suit	18.1 cm, 3.28°, 3.69°, –	22.0 cm, 19.8°, 23.8°, –	16.8 cm, 3.91°, 3.69°, –
Nev, space suit	18.8 cm, 5.89°, 6.53°, 1.70 cm	22.6 cm, 19.9°, 24.6°, 2.65 cm	17.8 cm, 6.59°, 7.15°, 2.95 cm

Body and garment	Warp: hem, crumple, waist, belt	Blender's cloth	Newton
The engineer, work suit	19.6 cm, 3.10°, 3.62°, –	23.9 cm, 17.8°, 24.8°, –	17.3 cm, 3.54°, 3.52°, –
The engineer, space suit	20.1 cm, 5.66°, 6.36°, 2.19 cm	24.0 cm, 18.1°, 21.4°, 3.08 cm	20.4 cm, 6.08°, 6.76°, 3.03 cm
The player, work suit	18.6 cm, 2.87°, 3.06°, –	23.2 cm, 13.8°, 15.2°, –	16.9 cm, 3.33°, 2.95°, –
The player, space suit	19.7 cm, 5.10°, 5.52°, 0.11 cm	21.5 cm, 17.9°, 20.7°, 4.17 cm	19.1 cm, 5.87°, 6.65°, 0.32 cm

Table G7 shows how a batch's time falls with the number of cards on the cluster. The same 24 Pixal3D takes, three at once on each L4, were run on one, two, four and eight nodes side by side. Every run used about the same card time, 210 to 221 minutes, and so cost about the same, while the time to the last result fell from 217 to 48 minutes. The four- and eight-node runs fell short of a linear speed-up for two reasons: only one zone had L4 cards, so their last nodes came 23 and 27 minutes after the order while the autoscaler kept retrying the other two zones, and eight nodes ran only eight jobs, so the run ended with its slowest job of 38 minutes.

Table G7. One batch on more cards: 24 Pixal3D takes, three at once on each L4, on a Scaleway cluster spanning Paris and Warsaw, 9 October 2026. Time runs from the submit to the last result; card time is the minutes the run's jobs held a card.

Nodes	Time	Speed-up	Last node came	Card time	Cost of card time
1	216.7 min	1.0	after 4.6 min	210 min	€2.76
2	118.9 min	1.8	after 4.2 min	215 min	€2.82
4	83.6 min	2.6	after 27 min	214 min	€2.81
8	48.0 min	4.5	after 23 min	221 min	€2.90

Tables G8 to G11 give the cost and time of October 2026 up to the provider's bill read on 10 October at 01:36 UTC. "Per stage" means here per kind of batch. The ledger records each batch's kind, its machines, their minutes and their price, but not the world, the place or the stage of a place's route that the batch served, so neither the cost of a world nor the cost of each stage of a place's route is recorded yet; only world 1 has been built, and Appendix B attributes its batches to places by their work folders. The tables cover October only, because the bill is read per month.

Up to the read, the ledger holds €547.31 of the bill's €627.74. Of the €80.43 between them, €51.87 is not compute: block storage for the machines' disks and one kept snapshot (€32.77), public IP addresses (€15.09), the image registry and the cluster control plane (€3.50), and object storage and secrets (€0.51). The other €28.56 is compute, and Table G11 traces it. €26.01 is processor machines that the ledger priced by the minute while the provider bills them by the started hour, almost all in rows written before the runner began pricing them by the hour on 8 October; €0.24 is two small machines that no batch rented; €2.28 is the 32-core 64 GB processor line, most likely the 47 engine runs on that type (€9.65 in the ledger), which the ledger priced by the minute and Table G11 does not price again by the started hour; and the last €0.03 is the card lines' differences netting out, from the L4 billed €4.97 above the ledger to the H100 PCIe billed €4.42 below it. Those card differences are not explained; a bill that lags the ledger and machines still running at the read are candidates.

The ledger's rows are never rewritten: a reconciliation file kept beside it tags as Pixal3D the 55 October rows (€91.68) written before that runner named its kind.

Table G8 adds the €80.43 to each kind of batch in proportion to its share of the ledger, since disks and addresses come with every machine and scale with its rented time; this is an allocation, not a measurement. Blender work and Pixal3D were about half of the month's spend, and the open judge a tenth. By what the work was for, €112.41 of the ledger was research and development on the framework itself: the game's engine runs, the container images and their proofs, the method experiments, and the benchmarks and comparisons that the rows name as such, among them the scale test of Table G7. The other €434.90 was the route's batches on world 1, reruns and rejected rounds included. It is not the cost of producing world 1: no place has yet been produced end to end and accepted beyond the hub, and the route's batches carry no mark when they were reruns made while the method changed, such as the hub's six rounds (Table C1). The cost of producing a place, stage by stage, is recorded only once each stage's output carries its own finish record.

Table G8. Cost and time per kind of batch, 1 to 10 October 2026. Batches and machine time from the cloud ledger; the column "with the rest of the bill" adds the €80.43 that the ledger does not hold in proportion to each kind's share. The median batch is the median wall time of the kind's batches. Blender covers settling, renders, review pages and close-up views; characters the person chain, rigging and drapes; engine runs the game engine's tests, shots, benchmarks and warm pool, whose machine types are read from their price; images and proofs the container images, their proofs and the kernel warm-up; the open picture models are FLUX.2 klein and Qwen-Image-Edit; the method experiments tried terrain, plants, clay and Infinigen. Processor machines are named by their cores.

Kind of batch	Batches	Machine time	Ledger	Share	With the rest of the bill	Median batch	Main card types
Blender	299	140.9 h	€155.42	28.4%	€178.26	12 min	L4 52%, 16-core 14%
Pixal3D	83	154.2 h	€130.98	23.9%	€150.23	40 min	L4 93%, H100 PCIe 3%
Open judge	82	16.1 h	€55.75	10.2%	€63.95	11 min	H100 PCIe 55%, two H100 SXM 37%
Library bake	238	50.7 h	€46.30	8.5%	€53.11	8 min	L4 84%, 16-core 8%
Characters	16	35.3 h	€35.31	6.5%	€40.50	45 min	L4 72%, H100 PCIe 28%
Parts and segments	119	26.3 h	€33.27	6.1%	€38.16	8 min	L4 53%, H100 PCIe 35%
Engine runs	142	39.3 h	€32.63	6.0%	€37.42	9 min	L4 70%, 32-core 64 GB 30%
Images and proofs	48	10.8 h	€24.06	4.4%	€27.60	10 min	16-core 44%, H100 PCIe 38%
Open picture models	63	6.0 h	€15.96	2.9%	€18.31	3 min	two H100 SXM 46%, H100 PCIe 38%
Method experiments	32	14.2 h	€15.74	2.9%	€18.05	33 min	L4 35%, 32-core 128 GB 28%
Sound	5	2.1 h	€1.88	0.3%	€2.16	5 min	L4 86%, H100 PCIe 10%

Kind of batch	Batches	Machine time	Ledger	Share	With the rest of the bill	Median batch	Main card types
All	1,127	495.9 h	€547.31	100%	€627.74		

Table G9. Rented machines by type, 1 to 10 October 2026, against the price the provider bills an hour. Rentals count the failed starts that were billed. The engine runs' rows name no machine; each is counted under the type whose price gives its cost by the minute (95 under the L4 and 47 under the 32-core 64 GB processor machine).

Machine	Rentals	Machine time	Cost	Per hour used	Billed an hour	Billed by
L4, 24 GB	1,080	396.3 h	€319.37	€0.81	€0.79	minute
H100 PCIe, 80 GB	148	26.0 h	€78.06	€3.00	€2.87	minute
16-core processor, 64 GB	64	20.5 h	€38.44	€1.87	€0.59	started hour
32-core processor, 64 GB	70	21.9 h	€31.79	€1.45	€0.85	started hour
two H100 SXM, 80 GB each	32	4.5 h	€31.55	€7.04	€6.62	minute
32-core processor, 128 GB	38	14.8 h	€28.73	€1.95	€1.18	started hour
L40S, 48 GB	45	10.3 h	€15.56	€1.52	€1.47	minute
P100, 16 GB	2	0.1 h	€2.44	€17.05	€1.23	started hour
16-core processor, 128 GB	2	1.6 h	€1.36	€0.84	€0.82	started hour
total		495.9 h	€547.31			

The L4 carried 58 percent of the ledger at about its billed price. The three processor types rented most cost 1.7 to 3.2 times their hourly price for each hour they were used, because the provider bills each started hour and their median rental in Blender batches was 12 to 33 minutes. Keeping such a machine for the next batch within the hour already paid, or sending a short batch to a type billed by the minute, would cut this waste. Table G10 gives the cost per unit of work for each kind of batch and card type.

Table G10. Cost per unit of work by kind of batch and card type, 1 to 10 October 2026, as the capacity report computes it from the same ledger. A unit is one take, render, question, part or

picture; seconds a unit is the median, and the cost of a unit is the whole rental of the machines that counted units, setup included, over those units. Rows written before 8 October record no units and are left out of the last three columns.

Kind	Card	Machines	Failed starts	Units	Seconds a unit	Cost a unit
Pixal3D	L4	251	32	52	1,358	€0.143
Pixal3D	H100 PCIe	4	14	19	867	€0.206
Blender	L4	220	35	486	116	€0.159
Blender	16-core processor	32	0	66	64	€0.304
Blender	32-core processor, 64 GB	19	90	34	117	€0.476
Library bake	L4	230	37	208	60	€0.061
Open judge	H100 PCIe	48	89	41	603	€0.597
Open judge	L40S	7	64	5	551	€0.299
Open judge	two H100 SXM	19	22	10	416	€1.357
Parts	L4	44	113	52	213	€0.056
Mesh parts	L4	5	15	7	1,059	€0.291
Mesh parts	H100 PCIe	4	25	8	1,080	€0.896
Segments	L4	28	22	63	58	€0.039
Pictures, FLUX.2 klein	L4	48	2	244	4	€0.002
Pictures, Qwen-Image-Edit	H100 PCIe	7	16	34	85	€0.093
Characters	L4	36	3	37	2,001	€0.625
Characters	H100 PCIe	2	1	1	1,285	€1.768
Rigging (UniRig)	L4	2	7	2	68	€0.033

Table G11. The provider's bill for October 2026, read on 10 October at 01:36 UTC, against the cloud ledger. The last column prices the ledger's processor machines again by the started hour, as the provider bills them; the engine runs' rows name no machine and are not priced again.

Bill line	Bill	Ledger	Ledger, processor machines by the started hour
L4, 24 GB	€324.34	€319.37	€319.37
H100 PCIe, 80 GB	€73.64	€78.06	€78.06
32-core processor, 128 GB	€50.74	€28.73	€50.74

Bill line	Bill	Ledger	Ledger, processor machines by the started hour
16-core processor, 64 GB	€41.30	€38.44	€41.30
32-core processor, 64 GB	€34.91	€31.79	€32.63
two H100 SXM, 80 GB each	€32.33	€31.55	€31.55
L40S, 48 GB	€14.27	€15.56	€15.56
P100, 16 GB	€2.45	€2.44	€2.45
16-core processor, 128 GB	€1.65	€1.36	€1.65
two small processor machines, no batch	€0.24		
compute	€575.87	€547.31	€573.32
block storage: machines' disks, one kept snapshot	€32.77		
public IP addresses	€15.09		
image registry, cluster control plane	€3.50		
object storage, secrets	€0.51		
all	€627.74	€547.31	